

When Do Structural Primers Help LLMs Reason over Graphs?

Baseline Accuracy and Answer Leakage

Gal Barak

github.com/GalBrk

Nitzan Zacharia

github.com/NitzanZacharia

Inbal Moryles

github.com/iinbal

Abstract

Prepending a primer of structural facts, such as each node’s degree, is a natural aid for a language model that answers questions about a graph given as text. We ask when such primers help, and why. Across 84,000 generations from Qwen3-1.7B and Qwen3-4B, with and without thinking, on six GraphQA tasks over 40-node Erdős–Rényi graphs, a solver that never sees the edges separates primers that state the answer from primers that supply side information. A primer that states the answer adds about 16 points where accuracy without it lies between 0.25 and 0.75, and it costs accuracy where the model is already correct. The responses show why: the model uses the primer as an alternative procedure. Qwen3-4B reads the stated degree instead of counting; it gains 27 points where its count fails and loses 12 where its count is 99% accurate. With thinking enabled, Qwen3-1.7B instead checks its count against the stated degree and gains 20 points. Side information acts mainly on false “yes” answers to edge queries: for Qwen3-1.7B, the richest primer removes 19 points of them, and a preamble that only restates the node set adds 18. Primer length does not order these effects.

1 Introduction

Fatemi et al. (2024) show that the accuracy of a large language model (LLM) on graph questions depends strongly on how the graph is written as text. A natural next step is to add derived structure to that text. We study *primers*: short preambles, placed before the graph encoding, that state each node’s degree, clustering coefficient or random-walk return probability, or the graph’s number of connected components. Graph neural networks take these statistics as input features (Dwivedi et al., 2022; Rampásek et al., 2022); here a pretrained LLM reads them as sentences.

Our research question is how primers change an LLM’s answers on graph tasks, and when they

help rather than harm. We divide it into three parts.

RQ1: how much of a primer’s effect comes from stating the answer, and how much from side information that does not determine it? **RQ2:** what decides whether a primer helps or harms a given model on a given task? **RQ3:** what does a primer change in the procedure the model follows?

Two controls make these questions answerable. A graph-blind solver reads the primer but never the edge list; where it is correct, the primer states the answer, and a gain may reflect retrieval rather than reasoning (Geirhos et al., 2020). A preamble that only restates the node set, together with primers of 37 to 4,691 characters, separates content from added text (Shi et al., 2023; Levy et al., 2024). We apply both controls to 84,000 generations and analyse the responses as well as their scores.

The results support one account (Section 5): a primer offers the model an alternative procedure, and its effect is the accuracy of the procedure the model adopts minus that of the one it abandons. This explains why effects follow the model’s accuracy without a primer (Section 4.1), why one primer helps and harms the same model on the same task (Section 4.2), and why a stated answer helps only a model that can use it (Section 4.3). On edge queries, side information instead changes how often the model answers “yes” (Section 4.4).

2 Related Work

GraphQA (Fatemi et al., 2024), NLGraph (Wang et al., 2023), GPT4Graph (Guo et al., 2023) and GraphArena (Tang et al., 2025) evaluate LLMs on graph problems and vary the encoding, the prompt and the task. Sanford et al. (2024) show that four of our six tasks can be solved by a single transformer layer, yet prompted LLMs trail trained transformers on them. Structure has been added to the input as an instruction to build the graph first (Wang et al., 2023), as motif statistics (Das et al., 2024) and as learned soft tokens (Perozzi et al., 2024).

Condition	States	Chars	Carries
none	no primer	–	–
components	component count	37	–
degree	degree	891	deg., count
clustering	clustering coefficient	1,629	–
rwse	return prob., 2 and 3 steps	2,949	–
all	the three rows above	4,691	deg., count
filler	“Node i is simply present”	1,829	–

Table 1: The seven conditions. *States*: what each primer gives per node (components: for the graph). *Chars*: mean primer length in characters. *Carries*: tasks with variable gold on which the graph-blind solver scores at least 0.85 (Appendix Table 5); abbreviations as in Table 2.

Huang et al. (2024) find that such text helps node classification through label-relevant phrases rather than through structure. These studies report gains without separating text that determines the answer from text that does not; we make that separation for every task and condition. Our primers render, as sentences, structural encodings that graph networks use as input features: in the taxonomy of Rampásek et al. (2022), degree, triangle-based clustering and random-walk return probability (Dwivedi et al., 2022) are local encodings, and the number of components is global.

The graph-blind solver is a partial-input baseline, as hypothesis-only models are for natural language inference (Gururangan et al., 2018; Poliak et al., 2018), applied here to an intervention rather than to a dataset. Added text can degrade reasoning (Shi et al., 2023; Levy et al., 2024), and accuracy depends on where information sits in the context (Liu et al., 2024) and on the order of a graph description (Ge et al., 2025).

3 Method

3.1 Primers and the graph-blind solver

A prompt consists of a primer, the graph in the incident encoding of Fatemi et al. (2024), and the question (Appendix A). Seven conditions share one renderer (Table 1). The primers write one sentence per node in id order, except components, which is a single sentence. filler writes “Node i is simply present within the graph G ” for each node: it restates the node set, which the encoding already lists, at a length within 12% of that of clustering.

The graph-blind solver reads the rendered primer and the graph’s node and edge counts, never the edge list. It applies exact rules where they exist and heuristic or fitted rules otherwise (fitted on

separate graphs), so its accuracy is a lower bound on what the primer alone determines. We call a primer *answer-carrying* on a task when the solver scores at least 0.85 there, and *side information* otherwise. The threshold does not matter: on the four tasks whose gold answer varies, degree and all score 1.00 on node_degree and edge_count, and every other primer scores at most 0.75 (Appendix Table 5).

3.2 Corpus, models and measurement

Graphs are Erdős–Rényi $G(n, p)$ graphs (Erdős and Rényi, 1959; Gilbert, 1959) with $n=40$ nodes at $p \in \{.10, .20, .35, .50\}$, 100 per density. The same graphs serve all six GraphQA tasks, and node_degree, connected_nodes and edge_existence query the same node. node_degree and edge_existence are extended to $p \in \{.65, .75, .85\}$. Four arms answer every prompt: Qwen3-1.7B and Qwen3-4B (Qwen Team, 2025) with thinking mode off (qwen3-1.7b, qwen3-4b) and on (-think), 84,000 generations in total. Decoding is greedy, with a budget of 8,192 new tokens (2,048 for the plain arms in the extension). For qwen3-1.7b on node_degree we use three further runs: 400 graphs per density, 1,600 graphs drawn with fresh seeds, and a grid that holds the mean degree at 8 or 16 while n ranges over $\{20, 40, 80, 160\}$.

Every task is scored by exact match; on connected_nodes, set-F1 would compress the contrasts about sixfold. On edge_existence the share of queried pairs that are edges rises from 10% to 85% with density, so we also report the hit rate (“yes” to an edge), the false-alarm rate (“yes” to a non-edge) and balanced accuracy. Each effect is a paired difference against none on the same graphs, in percentage points. A pair is dropped when either generation reaches the token budget (rates in Appendix Table 6). A *cell* is one (arm, task, density, primer) combination; we keep the 393 cells that retain at least 50 pairs, and the 47 dropped are edge_count cells. node_count and cycle_check have a constant gold answer at $n=40$, so we exclude them from all summaries (Appendix B). Tests are exact McNemar (McNemar, 1947) with Benjamini–Hochberg correction (Benjamini and Hochberg, 1995) within each (arm, task), and intervals are 95% bootstrap intervals over graphs (Efron, 1979). All but one of the 78,538 generations that terminate yield an answer.

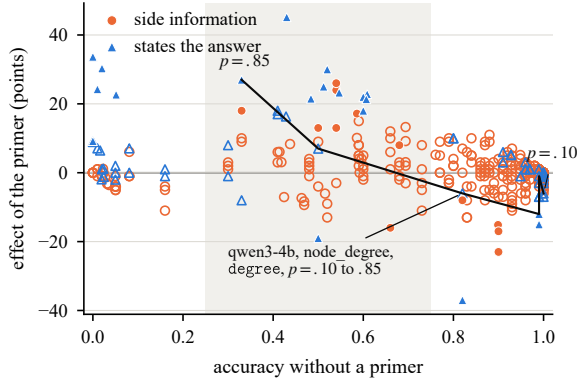


Figure 1: Effect of each primer against the cell’s accuracy without it. Triangles: answer-carrying; circles: side information; filled: significant after correction. The line joins the seven densities of degree for qwen3-4b on node_degree; shading marks baselines 0.25–0.75.

4 Results

4.1 Primer effects follow baseline accuracy

Figure 1 plots each cell’s effect against its accuracy without a primer, its *baseline*. Answer-carrying primers gain most at intermediate baselines: +16.0 and +16.7 points at baselines of 0.25–0.50 and 0.50–0.75, against +7.8 below 0.25 and –5.7 between 0.75 and 0.90. Above 0.90 they cost 1.4 points on average, and every such cell of qwen3-4b loses accuracy (–1 to –15). Side information follows the same shape at a smaller scale: +2.3 and +3.9 in the two middle bands, –1.8 below 0.25 and –0.8 above 0.90. The pattern is not regression to the mean: binning each cell on half its graphs and measuring the effect on the other half reproduces every band to within 3 points.

Thinking mode moves most cells out of this range. It raises the median baseline from 59.0 to 94.9 on the 1.7B model and from 87.0 to 100.0 on the 4B model, leaving 40 of qwen3-1.7b’s 103 cells between 0.25 and 0.75, 35 of 90 for qwen3-1.7b-think, 10 of 110 for qwen3-4b and none for qwen3-4b-think, which no primer moves by more than 2.1 points on node_degree, edge_existence or connected_nodes.

Table 2 breaks the effects down by task. Answer-carrying primers raise node_degree on both 1.7B arms (+7.5 to +11.7) and lower it on qwen3-4b (–6.5, –8.0), and degree raises qwen3-4b on edge_count by 27.7. As side information, all helps qwen3-1.7b on edge_existence (+11.5), clustering is the only primer that helps qwen3-4b there (+4.0), and filler produces the

largest costs (–15.7, –12.5).

4.2 The sign follows the procedure the primer replaces

For qwen3-4b on node_degree, the degree primer states the queried degree and changes accuracy by –6, –1, –7 and –12 points at $p=.10$ to .50, by –6 at .65, by +7 at .75 and by +27 at .85 (Figure 1, line). The losses are not a cost of added text: at $p=.50$ the primer turns 13 correct answers into errors and corrects one, no generation truncates, and filler, which is longer, costs 2 points.

The responses show a change of procedure. We classify a node_degree response as *counting* when it restates the queried node’s neighbour list and as *retrieval* when it gives the answer without doing so (Appendix Table 4). Without a primer qwen3-4b counts in 98–100% of responses, so its baseline is the accuracy of counting: 99–100% up to $p=.50$, and 50% and 33% at $p=.75$ and .85. Under degree it retrieves in 44–74% of responses, retrieval is 73–98% accurate at every density, and the median response shortens from 273 to 34 tokens at $p=.50$. Retrieval is therefore less accurate than counting up to $p=.50$ and more accurate at $p=.75$ and .85, which is the sign of the effect at those densities. Under all, which states the degree among two other statistics, the model retrieves less often (16–51%) and less accurately (49–96%), and the primer lowers accuracy at every density, by up to 37 points.

The thinking arm uses the same primer differently. qwen3-1.7b-think almost never retrieves: from $p=.35$ on it still enumerates the neighbours in 76–98% of responses under degree, and it reports a discrepancy between its count and the primer in 26% of responses up to $p=.50$ and in 49% beyond, against 1% and 4% without a primer, and resolves about two thirds of these discrepancies correctly. The primer adds 19.9 points [16.3, 23.5] over the seven densities (156 items corrected, 29 broken) and 31.6 points above $p=.50$, without the loss at high baselines (+5.2 and –1.0 at $p=.10$ and .20). qwen3-1.7b, finally, rarely uses the stated value: it retrieves in at most 4% of responses, and from $p=.50$, where its baseline is 5–30%, its accuracy under degree stays within 2 points of its accuracy without a primer.

	qwen3-1.7b				qwen3-1.7b-think				qwen3-4b				qwen3-4b-think			
	exist.	deg.	conn.	count	exist.	deg.	conn.	count	exist.	deg.	conn.	count	exist.	deg.	conn.	count
none (acc.)	70.5	60.5	72.5	1.8	99.2	76.8	79.7	86.3	90.5	99.2	86.0	2.0	100.0	99.7	98.7	97.6
pairs	399	399	399	211	397	376	390	13	398	400	400	393	393	380	373	34
filler	-15.7	-0.5	-6.2	-0.4	-0.8	-3.5	+2.0	+3.4	-7.0	-0.5	-12.5	+1.2	+0.0	+0.0	+0.8	-2.0
components	-8.2	+1.8	+0.8	-1.4	-0.8	-1.0	+2.8	+7.4	-2.8	+0.2	+9.3	-1.3	+0.0	-0.3	+0.5	+1.7
clustering	+3.8	+3.0	+1.3	-0.4	-1.0	+1.0	+1.5	-8.7	+4.0	-0.5	+1.0	-1.3	+0.0	-0.3	+0.0	+1.7
rwse	+5.3	+4.0	+1.0	-0.8	-1.3	+2.5	-0.3	-4.0	-10.0	-2.3	+0.0	-1.5	+0.0	-2.1	+0.0	+0.0
degree	+4.0	+7.5	+3.5	+4.0	-2.3	+11.7	+7.4	-61.5	-4.3	-6.5	-3.0	+27.7	+0.0	+0.0	+0.0	-23.5
all	+11.5	+10.3	+2.0	+2.4	-2.0	+11.6	+6.1	-8.3	-9.8	-8.0	-2.7	+3.8	+0.0	-0.8	-0.5	-41.5

Table 2: Pooled effects against none (points), $p \leq .50$, 100 graphs per (task, density). Tasks: edge_existence (exist.), node_degree (deg.), connected_nodes (conn.), edge_count (count). First row: accuracy under none over terminated generations; *pairs*: fewest pairs in which neither generation truncated. **Bold**: exact McNemar, Benjamini–Hochberg over each column’s six contrasts, $q < .05$; columns with *italic* pair counts (below 100) are never bolded, since their survivors are selected on termination.

4.3 A stated answer helps only through a procedure the model can execute

On edge_count, degree states every degree, so the answer is half their sum. qwen3-4b already describes this route without a primer (96–100% of responses) but is exact on only 2% of graphs. Given the degrees it is exact on 30% of graphs: +23, +30, +24 and +34 points across the four densities, with 114 items corrected and 5 broken. qwen3-1.7b adopts the route (from 7–67% of responses to 99–100%) but rarely completes the forty-term sum correctly: 4.7% of its answers are exact, a gain of 4.0 points that does not survive correction, although its mean absolute error falls from 136 to 40 edges. Where the primer states the answer outright, accuracy under it measures how much of the answer each arm takes up: on node_degree 0.99, 0.86, 0.81 and 0.43 for qwen3-4b-think, qwen3-1.7b-think, qwen3-4b and qwen3-1.7b, against 0.99, 0.66, 0.80 and 0.39 without it.

4.4 Side information shifts false alarms and helps where counting fails

On edge_existence the plain arms answer “yes” to 97–100% of true edges under every condition, and 94–99% of their errors are false alarms, so primers change accuracy through the false-alarm rate alone. Pooled over the seven densities, all lowers the false-alarm rate of qwen3-1.7b from 0.51 to 0.33 (−18.6 points [−24.0, −13.1]), and degree, rwse and clustering lower it by 6 to 9 points; filler raises it to 0.69 (+18.3 [14.2, 22.7]) and components to 0.61. For qwen3-4b, clustering is the only primer that lowers the false-alarm rate (0.16 to 0.08) and the only one that raises accuracy on the task (+4.6

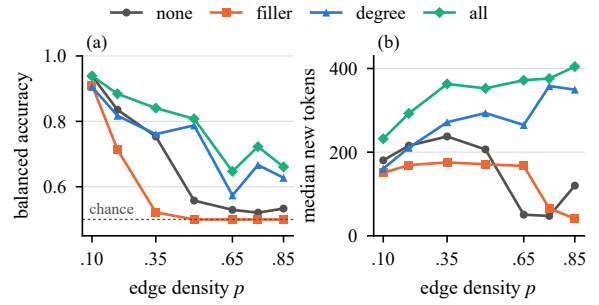


Figure 2: qwen3-1.7b on edge_existence: (a) balanced accuracy and (b) median new tokens by density. Without a primer the model answers “yes” to almost every pair from $p=.65$; under filler, from $p=.35$.

[2.0, 6.9]); the other five conditions raise false alarms by 7 to 14 points.

At high density qwen3-1.7b defaults to “yes” (Figure 2). Without a primer it answers “yes” to 98–99% of pairs from $p=.65$, in median responses of 48–120 tokens, at a balanced accuracy of 0.52–0.53. Primers that state per-node values keep the responses at 259–404 tokens, and the two that state degrees restore part of the balanced accuracy (degree +9.5 [4.7, 15.3], all +14.9 [8.4, 21.5]). filler brings the default forward to $p=.35$, where the model answers “yes” to 97% of pairs against 65% without a primer.

Side information also helps where counting fails. On node_degree above $p=.50$, where qwen3-4b counts correctly on 33–82% of items, clustering adds 11.3 points [6.3, 16.0] (46 items corrected, 12 broken) and rwse adds 7.7, whereas filler adds 2.0 (not significant). These primers state no degree, and the model keeps counting under them. For qwen3-1.7b the clustering gain replicates on the dedicated runs: +3.8 points [1.4, 6.1] at 400 graphs

per density, $+4.2$ $[2.0, 6.6]$ on 1,600 fresh graphs and $+6.2$ $[4.5, 8.0]$ across the 3,191 items of the fixed-degree grid ($p \leq .002$ in each), consistent with the $+3.0$ of the main sweep; its solver score equals that of none, so this gain is not leakage.

Some gains need no graph-specific content: components raises qwen3-4b on connected_nodes by $+9$ to $+11$ points at $p \geq .20$, where every graph is connected and the primer is the same 37-character sentence for all of them, and by $+7$ at $p=.10$, where it varies.

4.5 Bundling and length

all states every value that degree, clustering and rwse state, yet its effect is close to the mean of theirs ($+0.2$ points $[-1.1, 1.5]$ over 54 cells) and 4.6 points $[2.5, 6.8]$ below the best of them. Where the model must retrieve one value among three, the bundle loses what its parts gain: on qwen3-4b/node_degree above $p=.50$, degree, clustering and rwse add 9.3, 11.3 and 7.7 points, whereas all costs 21.3.

Primer length does not order the effects. Regressing each (arm, task)’s effects on the lengths of the five content primers gives slopes from -6.5 to $+3.4$ points per 1,000 characters, positive in 7 of 8 fits for the 1.7B model and in none of 8 for the 4B model, and filler lies 18.3 and 13.8 points below the fitted line where it harms most. On node_count, which the summaries exclude, qwen3-1.7b answers 39 instead of 40 on 68.5% of items without a primer; clustering removes this error almost entirely (1.2%), whereas the longer rwse leaves it (66.5%; Appendix Table 3).

5 Discussion and Conclusion

The results fit one account. A primer offers the model an alternative procedure, and its effect is the accuracy of the procedure the model adopts minus that of the one it abandons, weighted by how often it switches. Without a primer the model has to count, so baseline accuracy measures the procedure a primer can displace, which is why effects follow it. The account also explains each arm. qwen3-4b substitutes retrieval for counting, which costs accuracy where counting is reliable and gains where it fails; qwen3-1.7b-think keeps counting and uses the primer as a check, which gains without the loss; qwen3-1.7b rarely takes up the primer and gains little even where its count fails. A switch pays only if the new procedure can be executed: qwen3-4b

completes the degree sum and qwen3-1.7b does not, a failure of multi-step composition of the kind Dziri et al. (2023) describe. And a primer that makes retrieval harder, such as all, costs accuracy even where each of its parts helps.

Two results bear on prior work. node_degree and edge_count, solvable by a single transformer layer (Sanford et al., 2024), are where a stated answer helps and hurts most: for a prompted model the difficulty is the procedure it runs, not the information it holds. And added text acts through what it states — a preamble restating the node set raises false “yes” answers, one concrete form of the distraction Shi et al. (2023) report — while length alone does not order the effects (cf. Levy et al., 2024).

In answer to our questions: a stated answer is worth about 16 points at intermediate baselines and side information 2 to 4 (RQ1); the sign follows the accuracy of the procedure the primer displaces (RQ2); and a primer changes that procedure, to retrieval, to verification or, on edge queries, to a different rate of “yes” answers (RQ3). Evaluations of graph prompts should report each cell’s baseline accuracy, a graph-blind solver’s score and, for yes/no tasks, the false-alarm rate.

Limitations

We evaluate two model sizes from one family, with one encoding, template and primer position. Decoding is greedy, whereas the Qwen3 model card¹ recommends sampling in thinking mode, so the thinking arms’ truncation on edge_count (81% and 62%) is specific to this protocol. In our Erdős–Rényi graphs local clustering varies little within a graph from $p=.35$ on (standard deviation 0.05 there and less beyond), which limits what the clustering primer can convey. A cell of 100 graphs detects paired effects of about 8 points, so the side-information results rest on averages over cells. Procedures are classified by surface patterns, and a model may follow a procedure it does not state. The length comparison varies length only across primers whose content also differs.

Ethics Statement

This work evaluates open-weight models on synthetic graphs: no human subjects, personal data or annotation, and no direct risk of harm we can

¹<https://huggingface.co/Qwen/Qwen3-4B>

identify. Its main cost is GPU compute for about 217,000 generations.

References

- Yoav Benjamini and Yosef Hochberg. 1995. [Controlling the false discovery rate: A practical and powerful approach to multiple testing](#). *Journal of the Royal Statistical Society: Series B (Methodological)*, 57(1):289–300.
- Debarati Das, Ishaan Gupta, Jaideep Srivastava, and Dongyeop Kang. 2024. [Which modality should I use – text, motif, or image? : Understanding graphs with large language models](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 503–519.
- Vijay Prakash Dwivedi, Anh Tuan Luu, Thomas Laurent, Yoshua Bengio, and Xavier Bresson. 2022. [Graph neural networks with learnable structural and positional representations](#). In *International Conference on Learning Representations*.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D. Hwang, Soumya Sanyal, Sean Welleck, Xiang Ren, Allyson Ettinger, Zaid Harchaoui, and Yejin Choi. 2023. [Faith and fate: Limits of transformers on compositionality](#). In *Advances in Neural Information Processing Systems*, volume 36, pages 70293–70332.
- Bradley Efron. 1979. [Bootstrap methods: Another look at the jackknife](#). *The Annals of Statistics*, 7(1):1–26.
- Paul Erdős and Alfréd Rényi. 1959. [On random graphs I](#). *Publicationes Mathematicae Debrecen*, 6(3–4):290–297.
- Bahare Fatemi, Jonathan Halcrow, and Bryan Perozzi. 2024. [Talk like a graph: Encoding graphs for large language models](#). In *International Conference on Learning Representations*.
- Yuyao Ge, Shenghua Liu, Baolong Bi, Yiwei Wang, Lingrui Mei, Wenjie Feng, Lizhe Chen, and Xueqi Cheng. 2025. [Can graph descriptive order affect solving graph problems with LLMs?](#) In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 6404–6420.
- Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. [Shortcut learning in deep neural networks](#). *Nature Machine Intelligence*, 2(11):665–673.
- E. N. Gilbert. 1959. [Random graphs](#). *The Annals of Mathematical Statistics*, 30(4):1141–1144.
- Jiayan Guo, Lun Du, Hengyu Liu, Mengyu Zhou, Xinyi He, and Shi Han. 2023. [GPT4Graph: Can large language models understand graph structured data? An empirical evaluation and benchmarking](#). *arXiv preprint arXiv:2305.15066*.
- Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. 2018. [Annotation artifacts in natural language inference data](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 107–112.
- Jin Huang, Xingjian Zhang, Qiaozhu Mei, and Jiaqi Ma. 2024. [Can LLMs effectively leverage graph structural information through prompts, and why?](#) *Transactions on Machine Learning Research*.
- Mosh Levy, Alon Jacoby, and Yoav Goldberg. 2024. [Same task, more tokens: the impact of input length on the reasoning performance of large language models](#). In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15339–15353.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. [Lost in the middle: How language models use long contexts](#). *Transactions of the Association for Computational Linguistics*, 12:157–173.
- Quinn McNemar. 1947. [Note on the sampling error of the difference between correlated proportions or percentages](#). *Psychometrika*, 12(2):153–157.
- Bryan Perozzi, Bahare Fatemi, Dustin Zelle, Anton Tsitsulin, Mehran Kazemi, Rami Al-Rfou, and Jonathan Halcrow. 2024. [Let your graph do the talking: Encoding structured data for LLMs](#). *arXiv preprint arXiv:2402.05862*.
- Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger, and Benjamin Van Durme. 2018. [Hypothesis only baselines in natural language inference](#). In *Proceedings of the Seventh Joint Conference on Lexical and Computational Semantics*, pages 180–191.
- Qwen Team. 2025. [Qwen3 technical report](#). *arXiv preprint arXiv:2505.09388*.
- Ladislav Rampášek, Mikhail Galkin, Vijay Prakash Dwivedi, Anh Tuan Luu, Guy Wolf, and Dominique Beaini. 2022. [Recipe for a general, powerful, scalable graph transformer](#). In *Advances in Neural Information Processing Systems*, volume 35, pages 14501–14515.
- Clayton Sanford, Bahare Fatemi, Ethan Hall, Anton Tsitsulin, Mehran Kazemi, Jonathan Halcrow, Bryan Perozzi, and Vahab Mirrokni. 2024. [Understanding transformer reasoning capabilities via graph algorithms](#). In *Advances in Neural Information Processing Systems*, volume 37, pages 78320–78370.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. [Large language models can be easily distracted by irrelevant context](#). In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227.

Jianheng Tang, Qifan Zhang, Yuhua Li, Nuo Chen, and Jia Li. 2025. [GraphArena: Evaluating and exploring large language models on graph computation](#). In *International Conference on Learning Representations*.

Heng Wang, Shangbin Feng, Tianxing He, Zhaoxuan Tan, Xiaochuang Han, and Yulia Tsvetkov. 2023. [Can language models solve graph problems in natural language?](#) In *Advances in Neural Information Processing Systems*, volume 36, pages 30840–30861.

A Example prompt

A clustering-condition node_degree prompt at $p=0.10$ (elisions marked ...; line breaks added for width). Other conditions replace the first paragraph; none omits it.

Node 0 has clustering coefficient 0.20. Node 1 has clustering coefficient 0.00. ... Node 39 has clustering coefficient 0.33.

G describes a graph among nodes 0, 1, 2, ..., 38, and 39.

In this graph:

Node 0 is connected to nodes 2, 6, 8, 12, 35.

Node 1 is connected to nodes 5, 20.

...

Node 39 is connected to nodes 15, 16, 18, 20.

Q: What is the degree of node 16?

A:

B Constant-gold tasks

Arm	none	filler	comp.	clust.	rwse	degree	all
node_count							
1.7B	31.5	+39.8	+21.2	+67.3	+2.0	+6.8	+47.0
1.7B-think	98.6	+1.4	+0.9	+1.4	+1.4	+0.8	+1.4
4B	90.2	+9.3	+9.0	+8.8	+9.3	+8.0	+9.8
4B-think	99.5	+0.5	+0.0	+0.5	+0.5	+0.5	+0.5
cycle_check							
1.7B	99.7	+0.3	-0.6	+0.3	-7.5	-12.6	-1.1
1.7B-think	100.0	+0.0	+0.0	+0.0	+0.0	-0.7	+0.0
4B	99.7	-0.3	-1.3	+0.3	+0.3	-1.5	-0.5
4B-think	100.0	+0.0	+0.0	+0.0	+0.0	-0.8	+0.0

Table 3: The two tasks with constant gold at $n=40$, pooled over $p \leq .50$: accuracy under none (%) and paired effects against it (points). **Bold**: as in Table 2.

C Procedure on node_degree

p	.10	.20	.35	.50	.65	.75	.85
qwen3-4b							
acc., none	100	99	99	99	82	50	33
acc., degree	94	98	92	87	76	57	60
retrieves	44	56	74	62	71	64	69
acc. of retrievals	89	98	91	92	89	73	80
acc., all	93	96	92	84	45	31	25
retrieves	29	24	18	16	47	51	34
acc. of retrievals	90	96	78	88	60	49	65
qwen3-1.7b-think							
acc., none	93	95	60	60	52	47	42
acc., degree	98	94	84	82	78	76	87
enumerates	2	37	76	91	90	98	96
states a discrepancy	8	14	43	40	60	47	40
qwen3-1.7b							
acc., none	91	80	41	30	8	16	5
acc., degree	94	90	59	29	8	15	7
retrieves	4	2	0	0	0	0	4

Table 4: Procedure on node_degree by density (%; terminated generations). *Retrieves*: the response gives the answer without restating the queried node’s neighbour list. *Enumerates*: it lists the neighbours one per line. *States a discrepancy*: it reports that its count and the primer disagree. Without a primer every arm counts, so the none row is the accuracy of counting.

D Solver scores and truncation

Task	none	comp.	clust.	rwse	degree	filler	all
node_count	1.00	1.00	1.00	1.00	1.00	1.00	1.00
edge_count	.030	.030	.104	.030	1.00	.030	1.00
node_degree	.144	.144	.144	.210	1.00	.144	1.00
connected_nodes	.000	.000	.000	.001	.001	.000	.008
edge_existence	.735	.735	.727	.554	.746	.735	.741
cycle_check	1.00	1.00	1.00	1.00	1.00	1.00	1.00

Table 5: Accuracy of the graph-blind solver at $n=40$, averaged over seven densities (300 test graphs per density; fitted rules use separate graphs). It reads the rendered primer and the node and edge counts, never the edge list. **Bold**: answer-carrying (score ≥ 0.85) on a task whose gold varies. On node_count and cycle_check every score is 1.00 because the gold answer is constant at $n=40$.

Task	1.7B	1.7B-think	4B	4B-think
edge_count	22.4%	81.0%	0.6%	62.2%
cycle_check	6.1%	6.5%	1.0%	1.9%
node_degree	0.0%	1.1%	0.0%	1.6%
connected_nodes	0.0%	0.8%	0.0%	3.5%
edge_existence	0.0%	0.2%	0.1%	0.8%
node_count	0.0%	2.5%	0.0%	0.1%

Table 6: Share of main-sweep generations that reach the 8,192-token budget, by task and arm, pooled over condition and density. **Bold:** above 50%. In the high-density extension no (task, arm) exceeds 2.5%.